Unit 5 **[6 Hrs]**

Model Evaluation: Generalization Error, Out-of-Sample Evaluation Metrics, Cross Validation, Overfitting, Under Fitting and Model Selection, Prediction by using Ridge Regression, Testing Multiple Parameters by using Grid Search.

Model Evaluation: Model Evaluation is an integral part of the model development process. It helps to find the best model that represents our data and how well the chosen model will work in the future. Evaluating model performance with the data used for training is not acceptable in data science because it can easily generate overoptimistic and overfitted models. There are two methods of evaluating models in data science, Hold-Out and Cross-Validation. To avoid overfitting, both methods use a test set (not seen by the model) to evaluate model performance.

Hold-Out

In this method, the mostly large dataset is *randomly* divided to three subsets:

1. **Training set** is a subset of the dataset used to build predictive models.
2. **Validation set** is a subset of the dataset used to assess the performance of model built in the training phase. It provides a test platform for fine tuning model's parameters and selecting the best-performing model. Not all modelling algorithms need a validation set.
3. **Test set** or unseen examples is a subset of the dataset to assess the likely future performance of a model. If a model fit to the training set much better than it fits the test set, overfitting is probably the cause.

Cross-Validation

When only a limited amount of data is available, to achieve an unbiased estimate of the model performance we use *k*-fold cross-validation. In *k*-fold cross-validation, we divide the data into *k* subsets of equal size. We build models *k* times, each time leaving out one of the subsets from training and use it as the test set. If *k* equals the sample size, this is called "leave-one-out".

Model evaluation can be divided to two sections:

Classification: -

Confusion Matrix

A confusion matrix shows the number of correct and incorrect predictions made by the classification model compared to the actual outcomes (target value) in the data. The matrix is *N*x*N*, where *N* is the number of target values (classes). Performance of such models is commonly evaluated using the data in the matrix. The following table displays a 2x2 confusion matrix for two classes (Positive and Negative).

Confusion Matrix		Target			
		Positive	Negative		
Model	Positive	a	b	*Positive Predictive Value*	a/(a+b)

	Negative	c	d	*Negative Predictive Value*	d/(c+d)
		Sensitivity	*Specificity*	**Accuracy** = (a+d)/(a+b+c+d)	
		a/(a+c)	d/(b+d)		

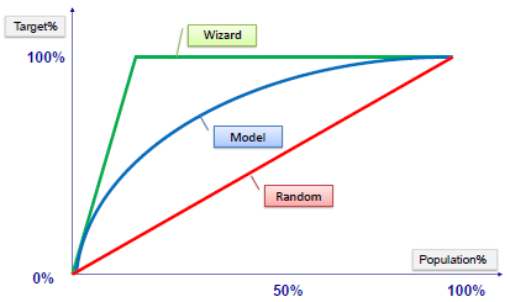
- **Accuracy** : the proportion of the total number of predictions that were correct.
- **Positive Predictive Value** or **Precision** : the proportion of positive cases that were correctly identified.
- **Negative Predictive Value** : the proportion of negative cases that were correctly identified.
- **Sensitivity** or **Recall** : the proportion of actual positive cases which are correctly identified.
- **Specificity** : the proportion of actual negative cases which are correctly identified.

Example:

Confusion Matrix		**Target**			
		Positive	Negative		
Model	Positive	70	20	*Positive Predictive Value*	0.78
	Negative	30	80	*Negative Predictive Value*	0.73
		Sensitivity	*Specificity*	**Accuracy** = 0.75	
		0.70	0.80		

Gain and Lift Charts

Gain or lift is a measure of the effectiveness of a classification model calculated as the ratio between the results obtained with and without the model. Gain and lift charts are visual aids for evaluating performance of classification models. However, in contrast to the confusion matrix that evaluates models on the whole population gain or lift chart evaluates model performance in a portion of the population.



Example:

Score Table

Target	Score
0	235
1	724
1	556
0	345
0	480
1	676
0	195
1	880
0	368
...	...

Sorted by Score

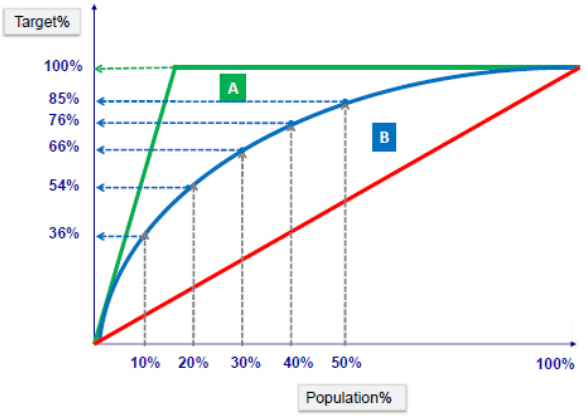
Target	Score
1	880
1	724
1	676
1	556
0	480
0	368
0	345
0	235
0	195
...	...

Gain Table

Count%	Target%
10	36
20	54
30	66
40	76
50	85
60	90
70	94
80	98
90	100
100	100

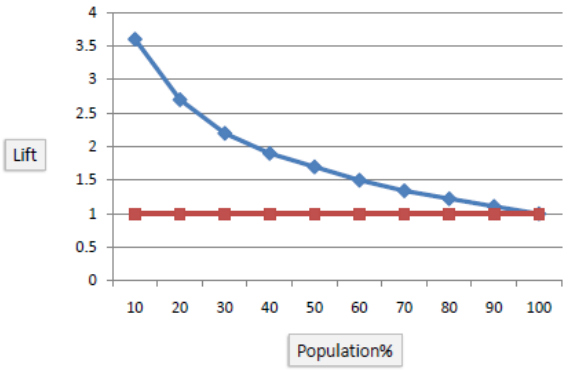
Lift Table

Count%	Lift
10	3.6
20	2.7
30	2.2
40	1.9
50	1.7
60	1.5
70	1.3
80	1.2
90	1.1
100	1



Lift Chart

The lift chart shows how much more likely we are to receive positive responses than if we contact a random sample of customers. For example, by contacting only 10% of customers based on the predictive model we will reach 3 times as many respondents, as if we use no model.



Regression:-

After building a number of different regression models, there is a wealth of criteria by which they can be evaluated and compared.

Root Mean Squared Error

RMSE is a popular formula to measure the error rate of a regression model. However, it can only be compared between models whose errors are measured in the same units.

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n}(p_i - a_i)^2}{n}}$$

a = actual target

p = predicted target

Relative Squared Error

Unlike RMSE, the relative squared error (RSE) can be compared between models whose errors are measured in the different units.

$$RSE = \frac{\sum_{i=1}^{n}(p_i - a_i)^2}{\sum_{i=1}^{n}(\overline{a} - a_i)^2}$$

Mean Absolute Error

The mean absolute error (MAE) has the same unit as the original data, and it can only be compared between models whose errors are measured in the same units. It is usually similar in magnitude to RMSE, but slightly smaller.

$$MAE = \frac{\sum_{i=1}^{n}|p_i - a_i|}{n}$$

Relative Absolute Error

Like RSE , the relative absolute error (RAE) can be compared between models whose errors are measured in the different units.

$$RAE = \frac{\sum_{i=1}^{n}|p_i - a_i|}{\sum_{i=1}^{n}|\overline{a} - a_i|}$$

Coefficient of Determination

The coefficient of determination (**R^2**) summarizes the explanatory power of the regression model and is computed from the sums-of-squares terms.

$$\text{Coefficient of Determination} \rightarrow \quad R^2 = \frac{SSR}{SST} = 1 - \frac{SSE}{SST}$$

$$\text{Sum of Squares Total} \rightarrow \quad SST = \sum (y - \bar{y})^2$$

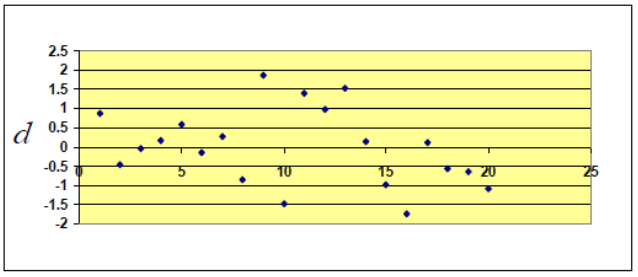
$$\text{Sum of Squares Regression} \rightarrow \quad SSR = \sum (y' - \bar{y}')^2$$

$$\text{Sum of Squares Error} \rightarrow \quad SSE = \sum (y - y')^2$$

R^2 describes the proportion of variance of the dependent variable explained by the regression model. If the regression model is "perfect", SSE is zero, and R^2 is 1. If the regression model is a total failure, SSE is equal to SST, no variance is explained by regression, and R^2 is zero.

Standardized Residuals (Errors) Plot

The standardized residual plot is a useful visualization tool in order to show the residual dispersion patterns on a standardized scale. There are no substantial differences between the pattern for a standardized residual plot and the pattern in the regular residual plot. The only difference is the standardized scale on the y-axis which allows us to easily detect potential outliers.



$$d_i = \frac{e_i}{S_e} \qquad e_i = y_i - y_i' \qquad S_e = \sqrt{\frac{SSE}{n-k-1}}$$

n = number of observations
k = number of predictors

Underfitting and Overfitting:

When we talk about the Machine Learning model, we actually talk about how well it performs and its accuracy which is known as prediction errors. Let us consider that we are designing a machine learning model. A model is said to be a good machine learning model if it generalizes any new input data from the problem domain in a proper way. This helps us to make predictions about future data, that the data model has never seen. Now, suppose we want to check how well our machine learning model learns and generalizes to the new data. For that, we have overfitting and underfitting, which are majorly responsible for the poor performances of the machine learning algorithms.

Before diving further let's understand two important terms:

- **Bias:** Assumptions made by a model to make a function easier to learn. It is actually the error rate of the training data. When the error rate has a high value, we call it High Bias and when the error rate has a low value, we call it low Bias.
- **Variance:** The difference between the error rate of training data and testing data is called variance. If the difference is high then it's called high variance and when the difference of errors is low then it's called low variance. Usually, we want to make a low variance for generalized our model.

Underfitting: A statistical model or a machine learning algorithm is said to have underfitting when it cannot capture the underlying trend of the data, i.e., it only performs well on training data but performs poorly on testing data. *(It's just like trying to fit undersized pants!)* Underfitting destroys the accuracy of our machine learning model. Its occurrence simply means that our model or the algorithm does not fit the data well enough. It usually happens when we have fewer data to build an accurate model and also when we try to build a linear model with fewer non-linear data. In such cases, the rules of the machine learning model are too easy and flexible to be applied to such minimal data and therefore the model will probably make a lot of wrong predictions. Underfitting can be avoided by using more data and also reducing the features by feature selection.
In a nutshell, Underfitting refers to a model that can neither performs well on the training data nor generalize to new data.

Reasons for Underfitting:

1. High bias and low variance
2. The size of the training dataset used is not enough.
3. The model is too simple.
4. Training data is not cleaned and also contains noise in it.

Techniques to reduce underfitting:

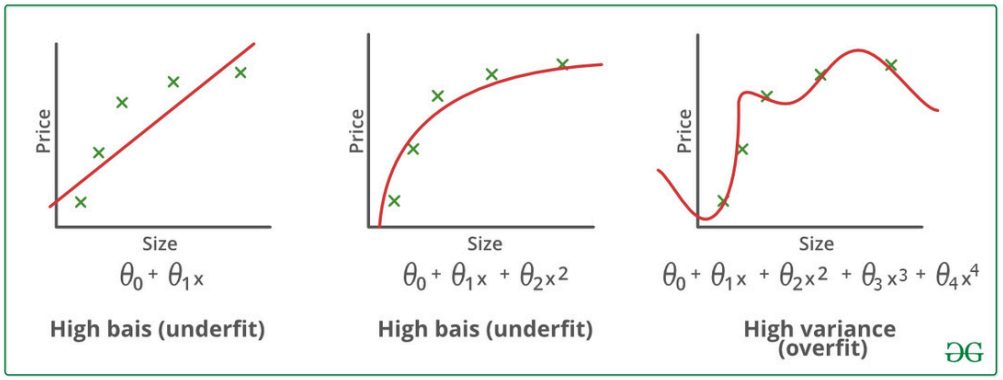
1. Increase model complexity
2. Increase the number of features, performing feature engineering
3. Remove noise from the data.
4. Increase the number of epochs or increase the duration of training to get better results.

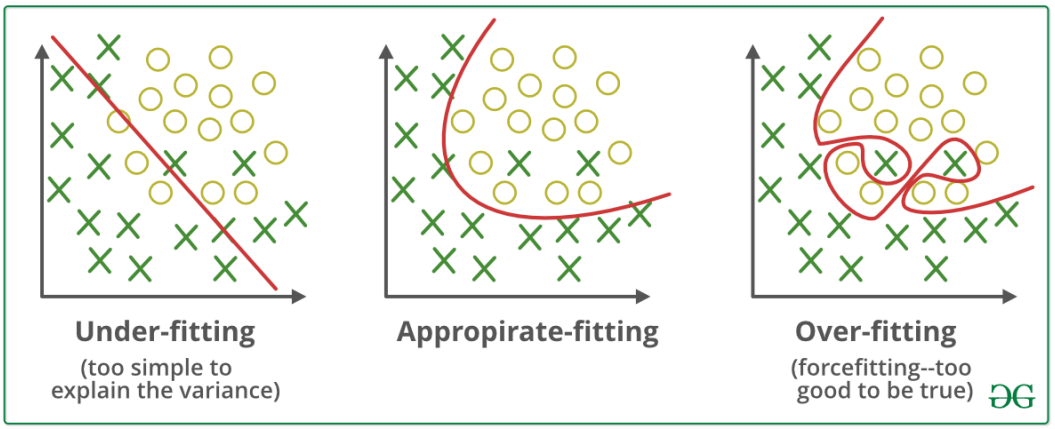
Overfitting: A statistical model is said to be overfitted when the model does not make accurate predictions on testing data. When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set. And when testing with test data results in High variance. Then the model does not categorize the data correctly, because of too many details and noise. The causes of overfitting are the non-parametric and non-linear methods because these types of machine learning algorithms have more freedom in building the model based on the dataset and therefore, they can really build unrealistic models. A solution to avoid overfitting is using a linear algorithm if we have linear data or using the parameters like the maximal depth if we are using decision trees.
In a nutshell, Overfitting is a problem where the evaluation of machine learning algorithms on training data is different from unseen data.

Reasons for Overfitting are as follows:

1. High variance and low bias
2. The model is too complex
3. The size of the training data

Examples:





Techniques to reduce overfitting:

1. Increase training data.
2. Reduce model complexity.
3. Early stopping during the training phase (have an eye over the loss over the training period as soon as loss begins to increase stop training).
4. Ridge Regularization and Lasso Regularization
5. Use dropout for neural networks to tackle overfitting.

Good Fit in a Statistical Model: Ideally, the case when the model makes the predictions with 0 error, is said to have a *good fit* on the data. This situation is achievable at a spot between overfitting and underfitting. In order to understand it, we will have to look at the performance of our model with the passage of time, while it is learning from the training dataset.
With the passage of time, our model will keep on learning, and thus the error for the model on the training and testing data will keep on decreasing. If it will learn for too long, the model will become more prone to overfitting due to the presence of noise and less useful details. Hence the performance of our model will decrease. In order to get a good fit, we will stop at a point just before where the error starts increasing. At this point, the model is said to have good skills in training datasets as well as our unseen testing dataset.

Select model

Different machine learning algorithms are searching for different trends and patterns. Consequently, one algorithm isn't the best across all datasets or for all use-cases. To find the best solution, we conduct a lot of experiments, evaluating different machine learning algorithms and tuning their hyper-parameters.

Here are some guidelines which are required to follow:

- Choose, justify and apply a model performance indicator (e.g. F1 score, true positive rate, within cluster sum of squared error, …) to assess your model and justify the choice of an algorithm
- Implement your algorithm in at least one deep learning and at least one non-deep learning algorithm, compare and document model performance
- Apply at least one additional iteration in the process model involving at least the feature creation task and record impact on model performance (e.g. data normalizing, PCA, …)

Depending on the algorithm class and data set size you might choose specific technologies / frameworks to solve your problem.

This chapter introduces various important topics:

- Train, Test & Validation Data
- Algorithm Exploration
- Hyper-parameter Optimization
- Ensembles

Terminology

Descriptive analytics

This is likely the most common type of analytics leveraged to create dashboards and reports. They describe and summarize events that have already occurred. For example, think of a grocery store owner who wants to know how items of each product were sold in all store within a region in the last five years.

Predictive analytics

This is all about using mathematical and statistical methods to forecast future outcomes. The grocery store owner wants to understand how many products could potentially be sold in the next couple of months so that he can make a decision on inventory levels.

Prescriptive analytics

Prescriptive analytics is used to optimize business decisions by simulating scenarios based on a set of constraints. The grocery store owner wants to creating a staffing schedule for his employees, but to do so he will have to account for factors like availability, vacation time, number of hours of work, potential emergencies and so on (constraints) and create a schedule that works for everyone while ensuring that his business is able to function on a day-to-day basis.

Supervised learning

Learn a model from labelled training data that allows us to make predictions about unseen or future data. We give to the algorithm a dataset with a right answer (label y), during the training, and we validate the model accuracy with a test data set with right answers. So, a data set needs to be split in training and test sets.

Classification

The problem is when we are trying to predict one of a small number of discrete-valued outputs. In other words, if our label is binary (binary classification) or categorical (multi-class classification).

Regression

When goal of the learning problem, is to predict continuous value output

Unsupervised learning

Giving a dataset, try to find tendency in the data, by using techniques like clustering.

Feature

Feature is an attribute used as input for the model to train. Other names include dimension or column.

Bias

Bias is the expected difference between the parameters of a model that perfectly fits your data and those your algorithm has learned. Low bias algorithms (Decision Trees, K-nearest Neighbors & Support Vector Machines) tend to find more complex patterns than high bias algorithms.

Variance

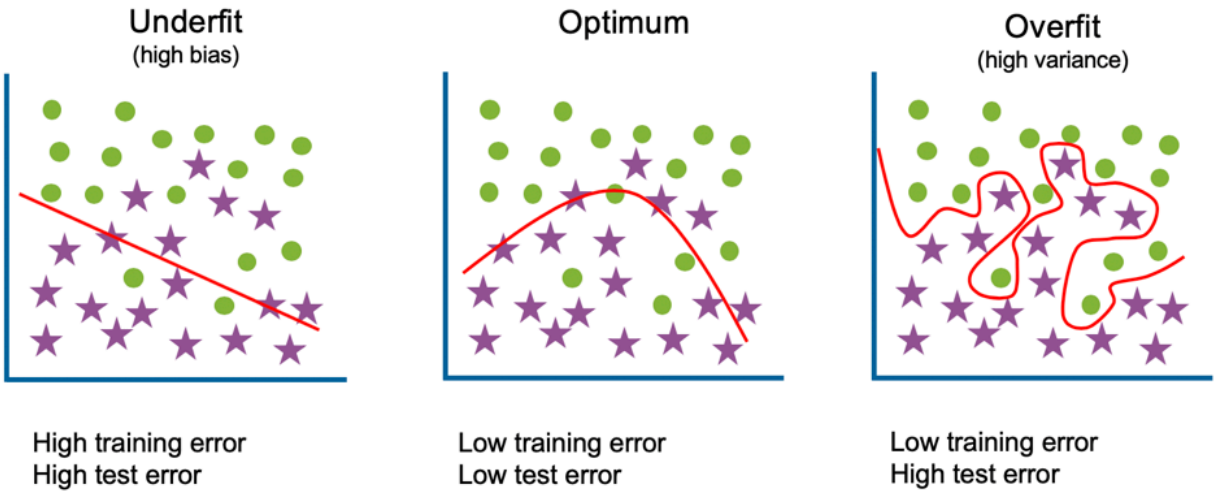
This is how much the algorithm is impacted by the training data; how much the parameters change with new training data. Low variance algorithms (e.g. Linear Regression, Logistic Regression & Naive Bayes) tend to find less complex patterns than high variance algorithms.

Underfitting

The model is too simple to capture the patterns within the data; this will perform poorly on data it has been trained on as well as unseen data. High bias, low variance. High training error and high test error.

Overfitting

The model is too complicated or too specific, capturing trends that do not generalize; it will accurately predict data it has been trained on but not on unseen data. Low bias, high variance. Low training error and high test error.



Bias-Variance Trade-off

The Bias-Variance Trade-off refers to finding a model with the right complexity, minimizing both the train and test error.

Further Reading: Bias-Variance Tradeoff Infographic

Train, Validation & Test

Machine learning algorithms learn from examples and, if you have good data, the more examples you provide, the better it will be at finding patterns in the data. However, you must be cautious of overfitting; overfitting is where a model can accurately make predictions for data it has been trained on but unable to generalize to data is hasn't seen before. This is why we split our data into training data, validation data and test data. Validation data and test data are often referred to interchangeably, however, they are described below as having distinct purposes. Training data This is the data used to train the model, to fit the model parameters. It will account for the largest proportion of data as you wish for the model to see as many examples as possible.

Validation data

This is the data used to fit hyper-parameters and for feature selection. Although the model never sees this data during training, by selecting particular features or hyper-parameters based on this data, you are introducing bias and again risk overfitting.

Test data

This is the data used to evaluate and compare your tuned models. As this data has not been seen during training nor tuning, it can provide insight into whether your models generalize well to unseen data.

Cross-Validation

The purpose of cross-validation is to evaluate if a particular algorithm is suited for your data and use-case. It is also used for hyper-parameter tuning and feature selection.

The data is split into train and validation sets (though you should have some test data put to one side too) and a model built with each of the slices of data. The final evaluation of the algorithm is the average performance of each of the models.

Hold-out Method

The simplest version of cross-validation is the hold-out method where we randomly split our data into two sets, a training set and a validation set. This is the quickest method as it only requires building a model once. However, with only one validation data set, there is a risk that it contains particularly easy, or difficult, observations to predict. Consequently, we could find we are overfitting to this validation data and on a test set it would perform poorly.

K-fold Cross-Validation

The k-fold cross-validation method involves splitting the data into k-subsets. You then train a model k times, each time using one of the k-subsets as its validation data. The training data will be all other observations not in the validation set. Your final evaluation is the average across all k folds.

Leave-one-out cross-validation

This is the most extreme version of K-fold cross-validation, where your k is N (the number of observations in your dataset). You train a model N separate times using all data except for one observation and then validate its accuracy with its prediction for that observation. Although you are thorough evaluating how well this algorithm works with your data set, this method is expensive as it requires you to build N models.

Stratified cross-validation

Stratified cross-validation enforces that the k-fold sets have similar proportions of observations for each class in either categorical features or the label.

Algorithm Exploration

The algorithms you explore should be driven by your use-case. By first identifying what you are trying to achieve, you can then narrow the scope of searching for solutions. Although not a complete list of possible methods, below are links that introduce algorithms for Regression, Classification, Clustering, Recommendations, and Anomaly Detection. A colleague and I also created this tool to help guide algorithm selection (Click here to visit Algorithm Explorer).

Regression

Regression algorithms are machine learning techniques for predicting continuous numerical values. They are supervised learning tasks which means they require labeled training examples.

Further reading: Machine Learning: Trying to predict a numerical value

Classification

Classification algorithms are machine learning techniques for predicting which category the input data belongs to. They are supervised learning tasks which means they require labeled training examples.

Further reading: Machine Learning: Trying to predict a classify your data

Clustering

Clustering algorithms are machine learning techniques to divide data into a number of groups where points in the groups have similar traits. They are unsupervised learning tasks and therefore do not require labeled training examples.

Further reading: Machine Learning: Trying to discover structure in your data

Recommendation Engines

Recommendation Engines are created to predict a preference or rating that indicates a user's interest in an item/product. The algorithms used to create this system find similarities between either the users, the items, or both.

Further reading: Machine Learning: Trying to make recommendations

Anomaly Detection

Anomaly Detection is a technique used to identify unusual events or patterns that do not conform to expected behavior. Those identified are often referred to as anomalies or outliers.

Further reading: Machine Learning: Trying to detect outliers or unusual behavior

Hyper-Parameter Optimization

Although the terms parameters and hyper-parameters are occasionally used interchangeably, we are going to distinguish between the two.

Parameters are properties the algorithm is learning during training.

For linear regression, these are the weights and biases; whilst for random forests, these are the variables and thresholds at each node.

Hyper-parameters, on the other hand, are properties that must be set before training.

For k-means clustering, you must define the value of k; whilst for neural networks, an example is the learning rate.

Hyper-parameter optimization is the process of finding the best possible values for these hyper-parameters to optimize your performance metric (e.g. highest accuracy, lowest RMSE, etc.). To do this, we train a model for different combinations of values and evaluate which find the best solution. Three methods used to search for the best combinations are Grid Search, Random Search, and Bayesian Optimization.

Grid Search

You specify values for each hyper-parameter, and all combinations of those values will be evaluated.

For example, if you wish to evaluate hyper-parameters for random forest, you could specify provide three options for the number of trees hyper-parameter (10, 20 and 50) and for the maximum depth of each tree you also provide three options (no limit, 10 and 20). This would result in a random forest model being built for each of the 9 possible combinations: (10, no limit), (10, 10), (10, 20), (20, no limit), (20, 10), (20, 20), (50, no limit), (50, 10), and (50, 20). The combination that provides the best performance will be those you use for your final model.

- **Pros:** Simple to use. You will find the best combination of the values you've provided. You can run each of the experiments in parallel.
- **Cons:** Computationally expensive as so many models are being built. If a particular hyper-parameter is not important, you are exploring different possibilities unnecessarily.

Random Search

You specify ranges or options for each hyper-parameter and random values of each are selected.

Continuing with the random forest example, you might provide the range for the number of trees to be between 10 and 50 and max_depth to be either no limit, 10 or 20. This time, rather than it compute all permutations, you can specify the number of iterations you wish to run. Say we only want five, then we might test something like (19, 20), (32 no limit), (40, no limit), (10, 20), (27, 10).

- **Pros:** Simple to use. More efficient and can outperform grid search when only a few hyper-parameters affect the overall performance. You can run each of the experiments in parallel.
- **Cons:** It involves random sampling so will only find the best combination if it searches that space.

Coarse to Fine

For both grid search and random search, you can also use the coarse to fine technique. This involves exploring a broader range of variables with wide intervals or all possible options. Once you've got your results from this initial search you explore the results to see if there are any patterns or particular regions that look promising. If so, you can repeat the process but refine your search.

For the random forest example above, we might notice that results are promising when the maximum depth has no limit and when the number of trees hyper-parameter is either 10 or 20. The search process is repeated but keeping the maximum depth hyper-parameter constant and increasing the granularity of the number of tree options, testing values of 12, 14, 16, 18, to see if we can find a better result.

- **Pros:** Can find more optimized hyper-parameters, improving the performance metric.
- **Cons:** Evaluation of the results to find the best regions to explore can be cumbersome.

Bayesian Optimization

Bayesian Optimization uses the prior knowledge of success with hyper-parameter combinations to choose the next best.

The technique uses a machine learning approach, building a model where the hyper-parameters are the features and the performance is the target variable. After each experiment, a new data point is added and a new model built.

It assumes similar combinations will have similar results and prioritize exploring regions where promising results have already been seen. However, it also takes into consideration uncertainty as a possibility for large gain; if there are large areas that have not yet been explored, it will also prioritize these as.

Taking just one hyper-parameter, the number of trees, the algorithm might first try 10 and get a pretty good performance. It then tries 32 and the performance is significantly better. Bayesian optimization builds a model based on the first two data points to predict performance. The model is likely to be linear with just the two data points so the next value chosen is 40, with the expectation that the performance will continue to improve as the number of trees increase. It does not. Now, it builds another model that suggests there might be improvement around 32 where it has seen the best result so far, however, there's still a large gap between 10 and 32 that hasn't been explored and due to large uncertainty, it chooses 21. Again, the model is tweaked with this new data and another value chosen…

- **Pros:** Can find more optimized hyper-parameters, improving the performance metric. It can reduce the time spent searching for an optimum solution when the number of parameters is high and each experiment is computationally expensive.
- **Cons:** You cannot run each experiment in parallel as the next combination of hyper-parameter values is determined by the prior runs. It also requires tuning — choosing a scale for each hyper-parameter and an appropriate kernel.

Ensemble Learning

Ensembles combine several machine learning models, each finding different patterns within the data, to provide a more accurate solution. These techniques can both improve performance, as they capture more trends, as well as reduce overfitting, as the final prediction is a consensus from many models.

Bagging

Bagging (bootstrap aggregations) is the method of building multiple models in parallel and average their prediction as the final prediction. These models can be built with the same algorithm (i.e. the Random Forest algorithm builds many decision trees) or you can build different types of models (e.g. a Linear Regression model and an SVM model).

Boosting

Boosting builds models sequentially evaluating the success of earlier models; the next model prioritizes learning trends for predicting examples that the current models perform poorly on. There are three common techniques for this: AdaBoost, Gradient Boosting and XGBoosted.

Stacking

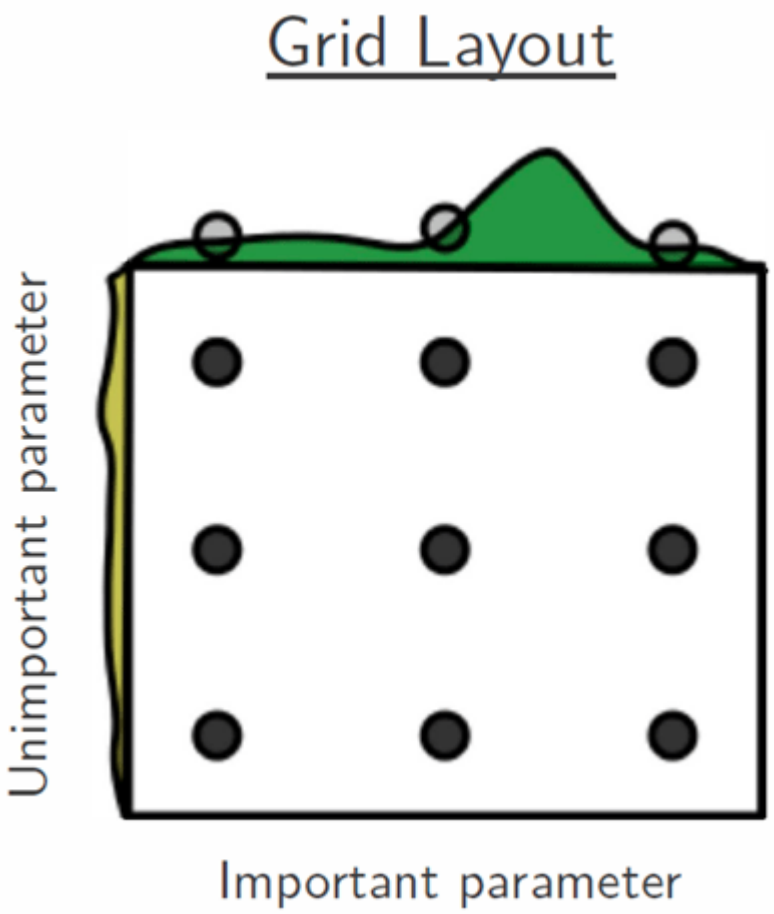
Stacking involves building multiple models and using their outputs as features into a final model. For example, your goal is to create a classifier and you build a KNN model as well as a Naïve Bayes model. Rather than choose between the two, you can pass their two predictions into a final Logistic Regression model. This final model may result in better results than the two intermediate models.

Testing Multiple Parameters by using Grid Search: -

Grid search

Grid search refers to a technique used to identify the optimal hyperparameters for a model. Unlike parameters, finding hyperparameters in training data is unattainable. As such, to find the right hyperparameters, we create a model for each combination of hyperparameters.

Grid search is thus considered a very traditional hyperparameter optimization method since we are basically "brute-forcing" all possible combinations. The models are then evaluated through cross-validation. The model boasting the best accuracy is naturally considered to be the best.



Grid layout.

Source

From the image above, we note that values are in a matrix-like arrangement.

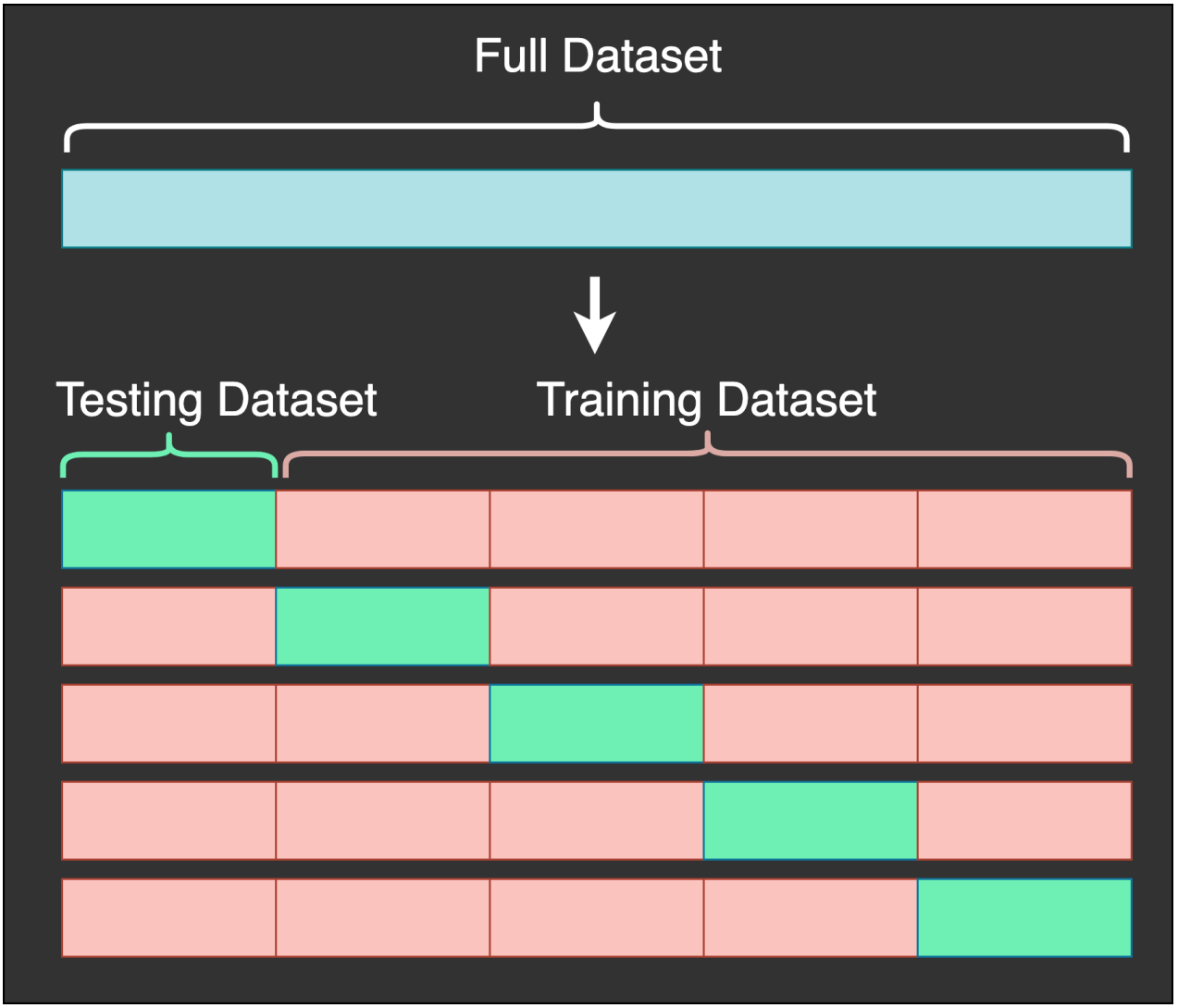
Cross validation

We have mentioned that cross-validation is used to evaluate the performance of the models. Cross-validation measures how a model generalizes itself to an independent dataset. We use cross-validation to get a good estimate of how well a predictive model performs.

With this method, we have a pair of datasets: an independent dataset and a training dataset. We can partition a single dataset to yield the two sets. These partitions are of the same size and are referred to as folds. A model in consideration is trained on all folds, bar one.

The excluded fold is used to then test the model. This process is repeated until all folds are used as the test set. The average performance of the model on all folds is then used to estimate the model's performance.

In a technique known as the k-fold cross-validation, a user specifies the number of folds, represented by k. This means that when k=5=5, there are 5 folds.



K-fold cross-validation with K as 5.

Source

Grid search implementation

The example given below is a basic implementation of grid search. We first specify the hyperparameters we seek to examine. Then we provide a set of values to test. After this, grid search will attempt all possible hyperparameter combinations with the aid of cross-validation. Let's break down this process into the steps below.

Steps

1. **Load dataset**.

My first step is loading the dataset using from sklearn.datasets import load_iris and iris = load_iris(). The iris dataset is sci-kit learn library in Python. Data is stored in a 150∗4150∗4 array. To see the contents of the dataset, we can use print(iris.data) and print(iris.data.shape).

To see the dataset contents, our input becomes:

```
from sklearn.datasets import load_iris
iris = load_iris()
print(iris.data)
print(iris.data.shape)
```

2. **Import GridSearchCV, svm and SVR.**

After loading the dataset, we then import GridSearchCV as well as svm and SVR from sklearn.model_selection as shown below.

GridSearchCV ensures an exhaustive grid search that breeds candidates from a grid of parameter values. As we shall see later on, these values are instanced using the parameter param_grid.

We import svm since the type of algorithm we seek to use is a support vector machine. The class SVR represents Epsilon Support Vector Regression. With this, the model has two free parameters; C and epsilon. We shall set parameters in the next step.

```
from sklearn.model_selection import GridSearchCV
from sklearn import svm
from sklearn.svm import SVR
```

3. **Set estimator parameters.**

In this implementation, we use the rbf kernel of the SVR model. rbf stands for the radial basis function. It introduces some form of non-linearity to the model since the data in use is non-linear. By this, we mean that the data arrangement follows no specific sequence.

```
estimator=SVR(kernel='rbf')
```

4. **Specify hyperparameters and range of values.**

We then specify the hyperparameters we seek to examine. When using the SVR's rbf kernel, the three hyperparameters to use are C, epsilon, and gamma. We can give each one several values to choose from.

Remember that it is possible to change these values and test them to see which values' collection gives better results. Below are my randomly chosen values.

```
param_grid={
        'C': [1.1, 5.4, 170, 1001],
        'epsilon': [0.0003, 0.007, 0.0109, 0.019, 0.14, 0.05, 8, 0.2, 3, 2, 7],
        'gamma': [0.7001, 0.008, 0.001, 3.1, 1, 1.3, 5]
    }
```

The param_grid parameter takes a list of parameters and ranges for each, as we have shown above.

5. **Evaluation.**

We mentioned that cross-validation is carried out to estimate the performance of a model. In k-fold cross-validation, k is the number of folds. As shown below, through cv=5, we use cross-validation to train the model 5 times. This means that 5 would be the k$_{\square}$ value.

scoring='neg_mean_squared_error' gives us the mean squared error. It is used in this form in grid search. This is meant to take the negative of the mean squared error to maximize and optimize it instead of minimizing the actual error.

n_jobs parameter specifies the number of concurrent processes that should be used for routines parallelized with the library joblib. In our case, at -1, it means that all CPUs are in use.

verbose gives us an option to produce logging information. We keep it at 0 to disable it since it may slow down our algorithm.

```
grid = GridSearchCV(

estimator=SVR(kernel='rbf'),
    param_grid={
        'C': [1.1, 5.4, 170, 1001],
        'epsilon': [0.0003, 0.007, 0.0109, 0.019, 0.14, 0.05, 8, 0.2, 3, 2, 7],
        'gamma': [0.7001, 0.008, 0.001, 3.1, 1, 1.3, 5]
    },
    cv=5, scoring='neg_mean_squared_error', verbose=0, n_jobs=-1)
```

6. **Fitting the data.**

We do this through grid.fit(X,y), which does the fitting with all the parameters.

All the code

Since we now understand the key aspects of the code, let's run all of the code.

You can access it and test it out here.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import GridSearchCV
from sklearn import svm
from sklearn.svm import SVR

iris = load_iris()
svc = svm.SVR()

grid = GridSearchCV(

estimator=SVR(kernel='rbf'),

param_grid={

'C': [1.1, 5.4, 170, 1001],

'epsilon': [0.0003, 0.007, 0.0109, 0.019, 0.14, 0.05, 8, 0.2, 3, 2, 7],

'gamma': [0.7001, 0.008, 0.001, 3.1, 1, 1.3, 5]
      },
      cv=5, scoring='neg_mean_squared_error', verbose=0, n_jobs=-1)

X = iris.data
y = iris.target

grid.fit(X,y)


#print the best parameters from all possible combinations
print("best parameters are: ", grid.best_params_)
```

Results:

```
best parameters are:
{'C': 170, 'epsilon': 0.0003, 'gamma': 0.008}
```

We are successful in our grid search and identify the best parameters being 170 from C, 0.0003 from the epsilon values, and gamma as 0.008.

Conclusion

Since grid search attempts all possible combinations, it becomes a computationally expensive method. We have defined grid search and explored how it works through a simple Python example.

There exist other optimization methods which vary in complexity and effectiveness. I hope to cover a few in the future.